



Advanced Terraform on AWS Lab6a

Module Foundations

Table of Contents

Expected Duration	1
Prerequisites.....	1
Phase 1. Consuming Existing Modules	1
Phase 2. Publishing Versioned Modules	7
Phase 3. Consuming Git-Hosted Modules	12
Phase 4. Promoting Module Versions	17

Expected Duration

Phase 1 of this lab is expected to take between 15-20 minutes

Phase 2 of this lab is expected to take 20-25 minutes

Phase 3 of this lab is expected to take 25-30 minutes

Phase 4 of this lab is expected to take 15-20 minutes

Total expected lab time is therefore 70-95 minutes

Try to avoid skipping straight to actionable lab steps as the logic of what you are trying to achieve may be lost and the lab learning value will therefore be reduced.

Prerequisites

This lab assumes the presence of a remote state backend, as created in Lab 4. If this does not exist in your lab environment then run Phase 2 of Lab 4 before continuing.

Phase 1. Consuming Existing Modules

Introduction

In the previous lab, you created two reusable Terraform modules:

- Network
- Compute

In this phase of the lab, we will initially take the role of a **module consumer** rather than a module author. Many Terraform users never create modules themselves. Instead, they consume modules developed and maintained by other teams. To do this successfully, they must understand how to:

- Discover module requirements
- Read module documentation
- Supply required inputs
- Consume module outputs
- Connect multiple modules together

The modules used in this phase already exist; they are the ones you created in Lab5. They and the associated tfvars files have been moved to a centralized location. Your task is to integrate them into a new root module by identifying and supplying the information they require.

Discovering Module Requirements

The starter configuration currently contains only three root-level variables:

- project_name
- env
- region

At present, no modules are referenced and no infrastructure will be created.

Initialise the Configuration

1. Change to the guided directory and initialise Terraform.

```
cd c:\qatfadvaws\lab6\guided
```

2. Review the files, update **providers.tf** with your S3 bucket details and initialize..

```
terraform init
```

Confirm the Starting State

3. Run planning...

```
terraform plan
```

Notice that no resources are proposed.

At this stage, the root module contains no infrastructure definitions and no module references.

Add Module References

The organisation has provided two reusable modules:

- Network
- Compute

For now, these modules are stored externally on the local filesystem at **c:\qatfadvaws\artifacts\local-modules**. Later in the lab they will be consumed from Git repositories.

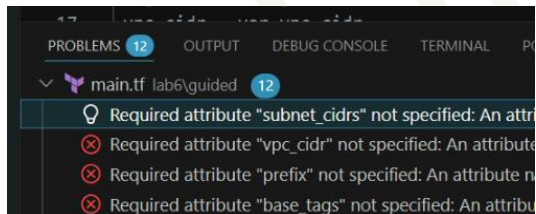
Enable the Module References

4. In main.tf, comment lines 1 and 9 to enable the following blocks:

```
module "network" {  
  source = "../../artifacts/local-modules/network"  
}  
  
module "compute" {  
  source = "../../artifacts/local-modules/compute"  
}
```

5. Run **terraform init --upgrade**

Terraform reports numerous errors. The modules require input variables that have not yet been supplied. You should see twelve required argument errors....



Pause for thought: How do you determine what information a module requires?

Use Module Documentation

6. The module author has provided documentation for each module. Review this at...

c:\qatfadvaws\artifacts\local-modules\network\README.md

c:\qatfadvaws\artifacts\local-modules\compute\README.md

Focus specifically on the documented:

- Inputs
- Outputs

You should identify the following required inputs:

Network Module

- prefix
- base_tags
- region
- vpc_cidr
- subnet_cidrs
- allow_groups
- security_group_rules

Compute Module

- prefix
- base_tags
- subnet_id
- security_group_id
- instances

Notice that some values are inputs, whilst others are intended to be supplied by outputs from another module.

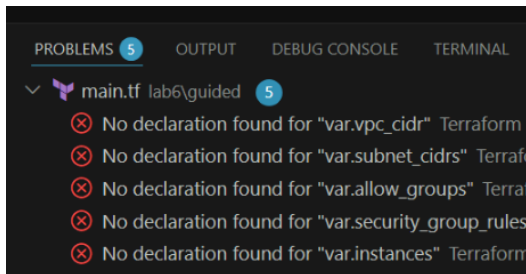
Supply Module Arguments

Enable the Full Module Definitions

7. In **main.tf**, uncomment lines **1** and **9** and comment lines **11** and **32** to enable the completed module definitions provided by the lab. The required arguments are now supplied.
8. Observe that several values are already available within the root module:
 - prefix
 - base_tags
 - region
9. Also notice that the Compute module receives the following from outputs exposed by the Network module.
 - app_subnet_id
 - security_group_id

This leaves the following values still undefined:

- vpc_cidr
- subnet_cidrs
- allow_groups
- security_group_rules
- instances

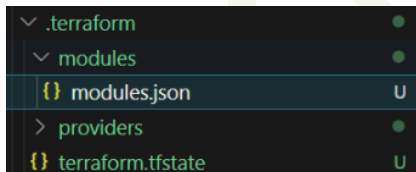


10. Re-initialize terraform and run planning again...

```
terraform init --upgrade
terraform plan
```

You should see errors relating to these missing values.

11. Note that a modules folder now appears under **.terraform** and a **modules.json** file identifies the module locations. Because these are on the local machine, the module files themselves are NOT copied into .terraform...



Enable Remaining Variables

12. In **variables.tf**, comment lines **31** and **64** to enable the provided variable definitions associated with:

- vpc_cidr
- subnet_cidrs
- allow_groups
- security_group_rules
- instances

These variables form part of the contract expected by the consumed modules. Note that there are no default values configured.

Supply Environment Values

13. Run:

```
terraform plan
```

No errors should be reported but Terraform should prompt you for variable values that have no default.

Press enter through each prompt without providing values.

14. The required values are to be supplied through provisioned tfvars files. These are the files created in Lab5 and are now at..

```
c:\qatfadvaws\artifacts\tfvars
```

15. Override those variables that have defaults and supply values for those that do not by running..

```
terraform plan --var-file="../../artifacts/tfvars/dev.tfvars"
```

The plan should now succeed.

Terraform has successfully:

- Located the modules
- Read the module contracts
- Accepted the supplied inputs
- Connected module outputs to downstream consumers
- Produced a valid deployment plan

Deploy to the Environments

16. Verify the new configuration is valid by deploying resources to the three environments..

```
terraform init -reconfigure -backend-config="key=dev.tfstate"
terraform plan --var-file="../../artifacts/tfvars/dev.tfvars"
terraform apply --var-file="../../artifacts/tfvars/dev.tfvars"

terraform init -reconfigure -backend-config="key=test.tfstate"
terraform plan --var-file="../../artifacts/tfvars/test.tfvars"
terraform apply --var-file="../../artifacts/tfvars/test.tfvars"

terraform init -reconfigure -backend-config="key=prod.tfstate"
terraform plan --var-file="../../artifacts/tfvars/prod.tfvars"
terraform apply --var-file="../../artifacts/tfvars/prod.tfvars"
```

Takeaway so far

When consuming a Terraform module, the most important artifacts are often not the Terraform files themselves, but the module documentation.

A module consumer should be able to determine:

- What inputs are required
- What outputs are exposed
- Which values must be supplied externally
- Which values can be obtained from other modules

Phase 2. Publishing Versioned Modules

Now we will stop treating shared-modules as simple folders and start treating them as versioned products. In enterprise Terraform, modules are deliverables: published, versioned, documented, and consumed by teams who never touch the module source.

Implementation

1. Create a **published-modules** folder in the course-wide **artifacts**

2. Create versioned paths:

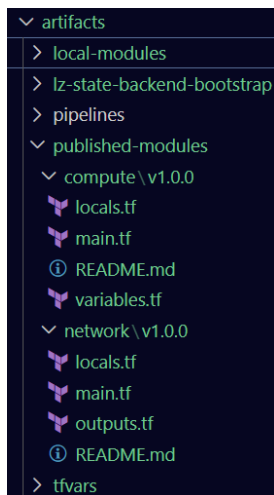
```
mkdir c:\qatfadvaws\artifacts\published-modules\network\v1.0.0
mkdir c:\qatfadvaws\artifacts\published-modules\compute\v1.0.0
```

3. Copy the **artifacts/local-modules** content into the appropriate version v1.0.0 folders, resulting in the file structure ...

```
copy c:\qatfadvaws\artifacts\local-modules\network\*
c:\qatfadvaws\artifacts\published-modules\network\v1.0.0

copy c:\qatfadvaws\artifacts\local-modules\compute\*
c:\qatfadvaws\artifacts\published-modules\compute\v1.0.0
```

4. Confirm the following folder structure now exists...



5. Update version details in **published-modules\compute\v1.0.0\main.tf**;

```
resource "terraform_data" "module_version" {
  input = {
    module   = "compute"
    version  = "1.0.0"
    change   = "compute module version 1.0.0. release"
  }
}
```

In a production environment further changes would be made to the compute module content. Here, this simple change will suffice as a module version change.

6. Update version details in **published-modules\network\v1.0.0\main.tf**;

```
resource "terraform_data" "module_version" {
  input = {
    module   = "network"
    version  = "1.0.0"
    change   = "network module version 1.0.0. release"
  }
}
```

In a production environment further changes would be made to the network module content. Here, this simple change will suffice as a module version change.

Modules are now treated as versioned artifacts rather than refactoring by-products.

Consume modules via “remote-style” sources

We will now update root to consume modules via versioned paths, not direct shared folders, “simulating remote consumption” without requiring Git or Terraform Registry. The point is the pattern: **pin to a version**.

9. Open **lab6/guided/main.tf**.

10. Locate the two active module blocks. They currently reference:

```
source = "../../../artifacts/local-modules/network"
source = "../../../artifacts/local-modules/security"
```

11. Update both module sources to point at the published, versioned folders:

```
module "network" {
  source = "../../../artifacts/published-modules/network/v1.0.0"
  ...
}

module "compute" {
  source = "../../../artifacts/published-modules/compute/v1.0.0"
  ...
}
```

12. Deploy v1.0.0 to DEV, expecting two changes...

```
terraform init -reconfigure -backend-config="key=dev.tfstate"
```

Note the updating of **modules/modules.json** in ./terraform

```
terraform plan --var-file="../../../artifacts/tfvars/dev.tfvars"
```

Two changes should be proposed

```
terraform apply --var-file="../../../artifacts/tfvars/dev.tfvars" --auto-approve
```

Two changes are made.

13. Deploy v1.0.0 to TEST and PROD, again expecting 2 changes...

```
terraform init -reconfigure -backend-config="key=test.tfstate"
```

```
terraform apply --var-file="../../../artifacts/tfvars/test.tfvars" --auto-approve
```

```
terraform init -reconfigure -backend-config="key=prod.tfstate"
```

```
terraform apply --var-file="../../../artifacts/tfvars/prod.tfvars" --auto-approve
```

You have moved from “modules as local folders” to “modules as versioned artifacts.” This is the same mindset used when consuming them from Git or Terraform Registry: **the root module pins a module version.**

Apply semantic versioning and upgrade safely

We will now demonstrate safe module upgrades with controlled version drift. We will create a module change, publish a new version, and upgrade DEV first—without forcing TEST or PROD to follow immediately.

This phase demonstrates controlled version drift: DEV upgrades first, while TEST and PROD remain pinned until an explicit upgrade is performed later

We will implement a small, safe v1.0.1 change to the security module contract, changing the manager tag value.

14. Create a new version folder for the network module:

artifacts/published-modules/network/v1.0.1

15. Copy network v1.0.0/* into v1.0.1/.

16. Update version details in **published-modules\network\v1.0.1\main.tf**;

```
resource "terraform_data" "module_version" {
  input = {
    module = "network"
    version = "1.0.1"
    change = "network module version 1.0.1. release"
  }
}
```

Remember that in a production environment, further changes would be made to the network module content.

17. Upgrade **DEV** to v1.0.1 of the network module by updating the **network** module version in **guided\main.tf** from **v1.0.0** to **v1.0.1**

```
12 module "network" {
13   source = "../../artifacts/published-modules/network/v1.0.1"
14   prefix = local.prefix
}
```

18. Run ...

terraform init -reconfigure -backend-config="key=dev.tfstate"

Note the switch to v1.0.1 of the newtork module...

```
See this backend unless the backend configuration changes:
Initializing modules...
- network in ../../artifacts/published-modules/network/v1.0.1
Initializing provider plugins...
```

Note the updating of **modules.json** in ./terraform/modules...

```
lab6 > guided > .terraform > modules > {} modules.json > ...
1 Source:"../../artifacts/published-modules/network/v1.0.1","Dir":"../../artifacts/published-modules/network/v1.0.1"]}]}
```



terraform plan --var-file="../../artifacts/tfvars/dev.tfvars"

```
# module.network.terraform_data.module_version will be updated in-place
~ resource "terraform_data" "module_version" {
  id      = "b35997dc-1d0b-acc9-7c78-d901b1d74cc5"
  ~ input = {
    ~ change = "Version 1.0.0. network module release" -> "Version 1.0.1. network module release"
    ~ version = "1.0.0" -> "1.0.1"
    # (1 unchanged attribute hidden)
  }
  ~ output = {
    - change = "Version 1.0.0. network module release"
    - module = "network"
    - version = "1.0.0"
  } -> (known after apply)
}

Plan: 0 to add, 1 to change, 0 to destroy.
```

terraform apply --var-file="../../artifacts/tfvars/dev.tfvars" --auto-approve

```
Apply complete! Resources: 0 added, 1 changed, 0 destroyed.

Outputs:

deployment_summary = {
  "environment" = "DEV"
  "modules" = {
    "compute" = "1.0.0"
    "network" = "1.0.1"
  }
}
```

19. Downgrade DEV to v1.0.0 of the network module by updating the **network** module version in **guided/main.tf** from **v1.0.1** to **v1.0.0** and then running ...

terraform init -reconfigure -backend-config="key=dev.tfstate"
terraform apply --var-file="../../artifacts/tfvars/dev.tfvars" --auto-approve

```
Apply complete! Resources: 0 added, 1 changed, 0 destroyed.

Outputs:

deployment_summary = {
  "environment" = "DEV"
  "modules" = {
    "compute" = "1.0.0"
    "network" = "1.0.0"
  }
}
```

Takeaway

Semantic versioning enables safe, incremental upgrades without forced global change. In this phase, module version changes were made by manually updating the module source in `main.tf`. This is intentional. Terraform resolves module sources during `terraform init`, which means module sources must be statically defined in configuration and cannot be selected dynamically via tfvars. Keeping module versions explicit in code makes structural changes visible, reviewable, and auditable, rather than hidden behind runtime variables.

In real-world environments, this manual step is not performed directly on production code. Instead, module version changes are introduced through Git workflows: developers update

the pinned version in a branch, validate the change in DEV, and promote it through TEST and PROD via pull requests and approvals. Phase 3 introduces Git-based module sources, where this same version pinning becomes part of a controlled promotion pipeline rather than a manual edit.

Phase 3. Consuming Git-Hosted Modules

Teaching Points

Now you take the next step: modules are no longer consumed locally. Instead, they are published to Git and consumed remotely, while multiple environments exist simultaneously. We will demonstrate how real platform teams distribute Terraform modules and consume them safely across environments *without* CI/CD pipelines or automation.

By the end of this part of the lab, you will understand how to:

- Treat Terraform modules as versioned products hosted in Git
- Consume modules from a remote Git repository using pinned versions
- Operate **DEV, TEST, and PROD simultaneously** using separate state files
- Keep naming and tagging contracts stable while changing distribution mechanisms
- Apply **Pattern A** for environment separation (parallel roots, parallel state)

We intentionally avoid automation here; promotion is manual so that module lifecycle concepts remain visible.

Pattern A – Parallel Environment Roots

In this course, **Pattern A** refers to an environment separation strategy where:

- Each environment (DEV, TEST, PROD) has its own Terraform root directory
- Each environment maintains its own independent remote state file
- All environments consume Terraform modules using explicitly pinned versions
- Promotion between environments is a conscious action, not an automatic side-effect

This pattern allows environments to exist **simultaneously**, with clear ownership of state and no hidden coupling. Platform teams commonly use it where multiple environments must remain live at all times.

In this lab, Pattern A is implemented using:

- Separate root folders (envs/dev, envs/test, envs/prod)
- Separate backend state keys (dev.tfstate, test.tfstate, prod.tfstate)

Pattern A in Context

There are several common ways teams separate Terraform environments.

Pattern A – Parallel Roots, Parallel State

Each environment has its own root configuration and its own state file. Environments exist simultaneously and are operated independently.

Pattern B – Single Root, Environment Switching

A single root configuration is reused with different variable files or workspaces to represent environments. Typically, only one environment is operated on at a time.

Pattern C – Branch-Based Environments

Each environment exists in a separate Git branch. This couples environment lifecycle to version control workflows and is discouraged for Terraform.

Why ? Branch-based environments are discouraged for Terraform because they couple long-lived stateful infrastructure to short-lived version-control workflows, increasing the risk of accidental promotion, state corruption, and unsafe rollback.

This lab uses **Pattern A** because it most closely reflects how platform teams operate long-lived DEV, TEST, and PROD environments in parallel, with explicit ownership of state and minimal hidden coupling.

Why platform teams prefer Pattern A

Pattern A avoids hidden coupling between environments. Each environment has explicit ownership of its own state, lifecycle, and failure domain. A broken DEV deployment cannot corrupt PROD state, and no environment is implicitly “current” based on workspace or branch context.

This Phase of the lab is organised into the following parts:

Part 1 - Reconfigure the lab file structure

Part 2 - Consume Git-hosted modules in DEV

Part 3 - Replicate for TEST and PROD

Part 4 - Validate the end state

Part 1 Reconfigure the lab file structure

We will first restructure our files in order to adopt **Pattern A Environment Separation** as discussed above. Following the next steps carefully as misconfiguration here will cause major knock-on effects.

1. At **lab6/guided**, create the following folder structure ..

```
envs/dev
envs/test
```

envs/prod

- Copy the following from **lab6/guided** into **each** envs **subfolder**...

main.tf
variables.t
providers.tf
local.tf
outputs.tf

- Delete **all** files and folders, except **/envs** from **lab6/guided**.
- From **c:\qatfadwavs\artifacts\tfvars**, copy the tfvars file into the appropriate /envs subfolder..

dev.tfvars into envs/dev

test.tfvars into envs/test

prod.tfvars into envs/prod

- Rename** all three tfvars files to **terraform.tfvars** ...



- Sanity Check** - Verify your folders match that shown above and that each **terraform.tfvars** declares the correct environment; dev, test, or prod.
- Update **providers.tf** in each to env folder, changing the state key file from **default.state** to...

envs/dev/providers.tf - **dev.tfstate**

envs/test/providers.tf - **test.tfstate**

envs/prod/providers.tf - **prod.tfstate**

Part 2 Create and Publish the Module Repositories (GitHub)

In this phase, you publish your modules to a GitHub repository. From this point onward, **Git** becomes the **distribution mechanism for modules**. Environments will no longer reference modules via local filesystem paths.

No infrastructure is deployed in this phase.

You will publish two separate repositories:

- terraform-aws-module-network
- terraform-aws-module-compute

Important rules (apply to both repositories):

- The module repository must contain only module code (no root configuration).
- Do not include provider blocks in the module.
- Do not include terraform { backend ... } blocks in the module.
- Tag the first release as v1.0.0

8. Download a Personal Access Token from your Github account.

9. Create the GitHub repository for the network module ...

In GitHub, create a new repository named: **terraform-aws-module-network**

Create the repository empty (do not add a README, .gitignore, or license).

10. Clone the repository locally (c:\module-repos is used throughout) ...

```
cd c:\
mkdir module-repos
cd module-repos
git clone {your network repo url}
cd terraform-aws-module-network
```

11. Add the repo folder to VSC Explorer; File > Add Folder to Workspace > c:\module-repos

12. Copy the **artifacts/published-modules/network/v1.0.0** files into your cloned repository ...

terraform-aws-module-network/

```
main.tf
locals.tf
variables.tf
outputs.tf
README.md
```

13. Create a repository **.gitignore** file with the following contents:

```
.terraform/
*.tfstate
*.tfstate.*
crash.log
```

This prevents accidental commits of local Terraform artifacts.

14. Set Github global settings (once only configuration change)

```
git config --global credential.helper manager
git config --global credential.useHttpPath true
```

15. Commit, tag, and push v1.0.0 of the network module...

```
cd c:\module-repos\terraform-aws-module-network
git config --local user.email "{your repo user email}"
git config --local user.name "{your repo username}"
git remote set-url origin https://{repo username}@github.com/{repo
username}/terraform-aws-module-network.git
```

```
git status
git add .
git commit -m "Publish module v1.0.0"
```

```
git tag v1.0.0
git push origin main
```

Supply your PAT when prompted

```
git push origin v1.0.0
```

16. In GitHub, confirm:

- The repository contains the module files at the repository root.
- The v1.0.0 tag exists under Tags/Releases.

17. Repeat steps 9-15 (skipping step 13) for the security module, using

terraform-aws-module-compute as the repository name, again cloning locally under **c:\module-repos** and using v.1.0.0 artifacts

At the end of this phase:

- Two independent module repositories exist in GitHub.
- Both repositories are tagged at v1.0.0.
- No infrastructure has been deployed yet.

Part 3 Consume Git-hosted modules in DEV

In this phase, you reconfigure the **DEV** environment to use Git-hosted modules. Dev should always be the first consumer of new/moved modules and versions. It is where contracts are validated before TEST or PROD are touched.

18. In **lab6/guided/env/dev/main.tf**, update the two module source references to match your repo username.

network module:

"git::https://github.com/{user}/terraform-aws-module-network.git?ref=v1.0.0"

security module:

"git::https://github.com/{user}/terraform-aws-module-compute.git?ref=v1.0.0"

19. Repeat this for **test/main.tf** and **prod/main.tf** (hint: main.tf is identical in all environments)
20. In **lab6/guided/envs/dev/providers.tf**, verify the backend key is **dev.tfstate**
21. At **lab6/guided/envs/dev**, run **terraform init**
22. Note that we no longer need to specify the backend config during initialization.
23. Note that the remote modules are pulled down from github to .terraform\modules
24. Create a plan **terraform plan**
25. Note that we no longer need to specify a tfvars as terraform.tfvars is read automatically.
26. This should be a no-op plan, given that all resources are currently deployed
27. Verify the new configuration produces no-ops for test and prod...

```
cd c:\qatfadvaws\lab6\guided\envs\dev
terraform init
terraform plan
```

```
cd c:\qatfadvaws\lab6\guided\envs\prod
terraform init
terraform plan
```

Both should result in no-ops

Phase 4. Promoting Module Versions

You have published Terraform modules to Git and consumed them remotely across multiple environments. Now you evolve a single module and promote that change across DEV, TEST, and PROD in a controlled and intentional way.

The focus is promotion discipline and version control. No automation is introduced.

This part of the lab is organised into five parts:

Part1 – Implement a Non-Breaking Network Module Change (v1.0.1)

Part 2 – Promote Network v1.0.1 to DEV

Part 3 – Promote to TEST and PROD

Part 1. Implement a Non-Breaking Network Module Change

In Part 1 you simulate a platform team releasing a non-breaking enhancement to a shared module and publishing it as a new version.

What “non-breaking” means here

You must **not** change the module interface in a way that would force consumers to edit their root modules.

Safe examples:

- Add a new output
- Add a new optional variable (with a default)
- Add an additional tag (derived internally)

Avoid:

- Renaming/removing variables
- Changing variable types
- Removing outputs
- Making an optional variable required

The new module version you will create introduces a non-breaking enhancement by updating a manually generated tag that identifies the module manager. No consuming environment needs to change its inputs.

1. Open **module-repos\terraform-aws-module-network\locals.tf**
2. Update the manager tag assigned to the module. This is the non-breaking change in this simplified demonstration..

```
manager = "John Robinson (NetMod v1.0.1)"
```

3. Push the change to github and tag it ...

```
cd c:\module-repos\terraform-aws-module-network
```

```
git status
```

```
git add locals.tf
```

```
git commit -m "Manager update"
```

```
git tag v1.0.1
```

```
git push origin main
```

```
git push origin v1.0.1
```

4. Verify in GitHub that the v1.0.1 tag exists for the network module.

Part 2. Promote Network Module v1.0.1

In this phase you promote the new network module v1.0.1 through DEV/TEST/PROD. The compute module remains pinned at v1.0.0.

5. In **lab6\guided\envs\dev\main.tf**, update the **network** module source reference:

Change `?ref=v1.0.0` to `?ref=v1.0.1`

6. At **lab6\guided\envs\dev**, re-initialise Terraform to fetch the new module version:

```
cd c:\qatfadvaws\lab6\guided\envs\dev
terraform init -upgrade
```

Review **.terraform/modules/modules.json** to verify the version update.

Review **.terraform/modules/network/locals** to verify it contains the updated manager tag

7. Create a plan using the DEV variable file:

```
terraform plan
```

Inspect the plan output and confirm that only network-managed resources show tag changes.

You should see changes similar to:

```
Changes to Outputs:
  ~ network-module-tags = {
    ~ manager = "Michael Coulling-Green (v1.0.0)" -> "John Robinson (v1.0.1)"
      # (2 unchanged attributes hidden)
  }
```

8. Apply the change:

```
terraform apply --auto-approve
```

Part 3. Promote to TEST and PROD

Repeating steps 5-8, with appropriate name adjustments, promote v1.0.1 of the network module to TEST and then PROD.

9. In the Console, confirm that network module is now pinned at v1.0.1 across DEV, TEST, and PROD and now include the tag: **manager = John Robinson (Network v1.0.1)** across the VPC, Route table and Security group.
10. Confirm the compute module remains unchanged and pinned at v1.0.0 with the tag: **manager = Michael Coulling-Green (SecMod v1.0.0)** across all EC2 instances.

Takeaways

- Modules can evolve safely without changing consumer contracts.
- Promotion is explicit and intentional.
- Independent repositories enable independent lifecycles.
- Terraform plan output is a reliable audit mechanism.
- State isolation enables safe multi-environment promotion.

11. Cleanup...

```
cd c:\qatfadvaws\lab6\guided\envs\dev  
terraform destroy --auto-approve
```

```
cd c:\qatfadvaws\lab6\guided\envs\test  
terraform destroy --auto-approve
```

```
cd c:\qatfadvaws\lab6\guided\envs\prod  
terraform destroy --auto-approve
```

***** Congratulations, you have completed this lab *****

Copyright

© 2026 QA Michael Coulling-Green. All rights reserved. These lab materials are provided as part of an authorised training course. Course attendees may reuse these materials for their own personal learning and reference outside of the training session. Unauthorised copying, redistribution, publication, or use of these materials to deliver training is prohibited without prior written permission from the author.

© QA 2026